
Unified Implicit Differentiation for Structured Deep Learning Layers: A Review and Extension

Yifan Zhang

zhangyf52025@shanghaitech.edu.cn

Abstract

Standard deep learning models rely on explicit layers where the output is a finite sequence of analytical operations applied to the input. Recent advances have introduced *implicit layers*, where the output is defined as the solution to a mathematical problem, such as a fixed-point equation, an ordinary differential equation (ODE), or a constrained optimization problem. While these approaches appear distinct, they share a common mathematical foundation: the reliance on implicit differentiation for efficient backpropagation. In this project, we provide a unified mathematical formulation that encompasses Deep Equilibrium Models (DEQ), Neural ODEs, and Differentiable Optimization layers (OptNet). We critically review literature in this domain and propose the **SVD-Free Newton-Schulz Layer**, a novel approach that improves upon existing implicit solvers by replacing expensive Spectral Decompositions (SVD) with iterative matrix multiplications and implicit Lyapunov differentiation. Preliminary experiments demonstrate that our method achieves superior numerical stability in degenerate cases and better computational scaling on GPUs compared to state-of-the-art spectral baselines.

1 Introduction

Modern deep neural networks are predominantly built upon explicit transformations (e.g., Conv2D, BatchNorm, ReLU), where the hidden state z_{l+1} is explicitly computed from z_l . However, a growing body of research suggests that defining layers *implicitly*—via the solution to an underlying equilibrium or optimization condition—offers significant advantages in terms of memory efficiency, parameter sharing, and inductive bias.

This project focuses on the unification of these implicit methods. Specifically, we investigate the connection between Deep Equilibrium Models [4], Neural ODEs [6], and Differentiable Optimization Layers [3, 1]. By framing these diverse architectures under a single root-finding problem $g(z, x, \theta) = 0$, we leverage the Implicit Function Theorem (IFT) to derive a generalized backpropagation algorithm.

The contributions of this report are threefold:

1. We present a unified mathematical formulation for implicit layers, showing that DEQs, Neural ODEs, and OptNet are special cases of a generic root-finding problem.
2. We provide a critical review of key literature, highlighting the computational bottleneck of the Jacobian inverse computation and the instability of spectral methods.
3. We propose the **SVD-Free Newton-Schulz Layer**, which computes matrix functions via iterative matrix multiplication and gradients via Lyapunov solvers. We validate its unconditional stability and efficiency on degenerate matrix tasks.

2 Unified Problem Formulation

Unlike explicit layers where $z = f(x, \theta)$, an implicit layer defines the output $z^* \in \mathbb{R}^d$ as the solution to a constraint equation. We formalize this unification below.

2.1 The Foundation of Mathematics: The Matrix and the Structured Gradient

2.1.1 Matrix Backpropagation for Deep Networks with Structured Layers

The seminal paper for matrix backpropagation [8] introduced a formal framework for training deep networks with global structured layers (e.g., SVD, eigenvalue decomposition) end-to-end.

This work established Matrix Backpropagation (MBP): A methodology to compute gradients for matrix functions using the invariant property of the Frobenius inner product (trace), generalizing the chain rule to matrix variables.

$$\begin{aligned} \frac{\partial L^{(l+1)}}{\partial Y} : dY &= \frac{\partial L^{(l+1)}}{\partial Y} : \mathcal{L}(dX) \triangleq \mathcal{L}^* \left(\frac{\partial L^{(l+1)}}{\partial Y} \right) : dX \\ &\Rightarrow \mathcal{L}^* \left(\frac{\partial L^{(l+1)}}{\partial Y} \right) = \frac{\partial L^{(l)}}{\partial X} \end{aligned} \quad (1)$$

They also applied this framework to spectral layers, and derived the specific gradients for layers involving SVD ($X = U\Sigma V^T$) and symmetric EIG ($X = U\Sigma U^T$).

2.1.2 A Unified Framework for Matrix Backpropagation

This paper [7] revisits the Ionescu approach, corrects it, and proves its theoretical equivalence to the Daleckii-Krein/Bhatia (DK/B) formula, arguing that DK/B is superior for implementation.

This work generalized DK/B Formula: For a diagonalizable matrix $X = U\Lambda U^{-1}$ and $Y = f(X)$:

$$\frac{\partial \phi}{\partial X} = U^{-T} [f^{[1]}(X) \odot (U^T \frac{\partial \phi}{\partial Y} U^{-T})] U^T$$

where

$$[f^{[1]}(\mathbf{X})]_{ij} = \begin{cases} \frac{f(\lambda_i) - f(\lambda_j)}{\lambda_i - \lambda_j}, & i \neq j \\ f'(\lambda_i), & \text{otherwise.} \end{cases}$$

And for the SPD Case, since $U^{-1} = U^T$, this simplifies to:

$$\frac{\partial \phi}{\partial X} = U [f^{[1]}(X) \odot (U^T \frac{\partial \phi}{\partial Y} U)] U^T$$

Darley and Bonnet (2025) provide this unified framework that corrects theoretical inaccuracies and proves the equivalence of the original method to the Daleckii-Krein/Bhatia (DK/B) formula. The Unified approach addresses the critical issue of numerical instability in Ionescu's formulation where close eigenvalues lead to gradient explosion by utilizing the DK/B formula with Loewner matrices to robustly handle degenerate cases.

The experiments can be seen in Appendix A.

2.2 Optimization as Model: From Constraint Networks to Embedded Solvers

2.2.1 OptNet: Differentiable Quadratic Programs as Layers

Amos and Kolter [3] introduced the OptNet architecture, which integrates optimization problems specifically Quadratic Programs (QPs) as individual layers within deep networks. Unlike standard layers with explicit analytical forms, the output of an OptNet layer is the solution to a constrained optimization problem.

Formally, the layer output z_{i+1} is defined as the minimizer of a QP:

$$\begin{aligned} z_{i+1} = \underset{z}{\operatorname{argmin}} \quad & \frac{1}{2} z^T Q(z_i) z + q(z_i)^T z \\ \text{subject to} \quad & A(z_i) z = b(z_i) \\ & G(z_i) z \leq h(z_i) \end{aligned} \quad (2)$$

where the problem parameters $\{Q, q, A, b, G, h\}$ depend on the previous layer z_i .

To enable end-to-end training, the authors derived the gradients of the optimal solution with respect to the input parameters by differentiating the KKT conditions. By applying matrix differentials to the KKT optimality conditions, the gradients are obtained by solving the following linear system:

$$\begin{bmatrix} Q & G^T & A^T \\ D(\lambda^*)G & D(Gz^* - h) & 0 \\ A & 0 & 0 \end{bmatrix} \begin{bmatrix} dz \\ d\lambda \\ d\nu \end{bmatrix} = \begin{bmatrix} -dQz^* - dq - dG^T\lambda^* - dA^T\nu^* \\ -D(\lambda^*)dGz^* + D(\lambda^*)dh \\ -dAz^* + db \end{bmatrix} \quad (3)$$

Here, $D(\cdot)$ creates a diagonal matrix, and ν^*, λ^* are the optimal dual variables. Crucially, the matrix on the left-hand side is the KKT matrix of the QP. Since this matrix is already factored during the forward pass (using a primal-dual interior point method), the backward pass gradients can be computed with virtually no additional cost by reusing the factorization.

I conducted a preliminary experiment based on the research framework outlined in the paper, with detailed procedures available in Appendix B.

2.2.2 Differentiable Convex Optimization: From QP to Cone Programs

Agrawal et al. [1] generalized the concept of differentiable layers from QPs to the broader class of disciplined convex programs (DCP). They proposed a framework where a parametrized problem is represented in *Affine-Solver-Affine* (ASA) form, composed of an affine map from parameters to cone problem data, a conic solver, and an affine retrieval map.

The solution map $\mathcal{S}(\theta)$ is differentiated using the chain rule:

$$D^T \mathcal{S}(\theta) = D^T C(\theta) D^T s(A, b, c) D^T R(\tilde{x}^*) \quad (4)$$

where C is the canonicalization map and s is the cone solver. To compute $D^T s(A, b, c)$, the authors implicitly differentiate the homogeneous self-dual embedding of the cone program. Specifically, the derivative of the solver with respect to the data matrix Q (constructed from A, b, c) is derived from the normalized residual map $\mathcal{N}$:

$$Ds(Q) = -(D_z \mathcal{N}(s(Q), Q))^{-1} D_Q \mathcal{N}(s(Q), Q) \quad (5)$$

When the linear system involved in this derivative is singular (at non-differentiable points), a least-squares solution is computed instead.

2.2.3 Input Convex Neural Networks: Inference via Optimization

In contrast to embedding optimization as a layer, Amos et al. [2] proposed Input Convex Neural Networks (ICNNs), scalar-valued networks $f(x, y; \theta)$ designed to be convex with respect to inputs y . This convexity allows for efficient inference by solving the global optimization problem $\operatorname{argmin}_y f(x, y; \theta)$.

To ensure convexity, the network architecture imposes constraints on the weights and activation functions. A fully input convex network (FICNN) follows the recurrence:

$$z_{i+1} = g_i \left(W_i^{(z)} z_i + W_i^{(y)} y + b_i \right) \quad (6)$$

where $W_i^{(z)}$ must be non-negative and g_i must be convex and non-decreasing.

2.3 Depth as Time: Continuous and Infinite Depth Models

While the approaches discussed in Section 2 view layers as discrete constrained optimization steps, another paradigm shifts the perspective of "depth" toward continuous time integration or infinite iterations.

2.3.1 Neural Ordinary Differential Equations

Neural ODEs [6] challenge the traditional residual network formulation. A residual block $z_{t+1} = z_t + f(z_t, \theta_t)$ can be viewed as a discrete Euler discretization of a continuous transformation. By taking the limit as the step size approaches zero, the discrete sequence of layers transforms into continuous dynamics governed by an ordinary differential equation (ODE):

$$\frac{dz(t)}{dt} = f(z(t), t, \theta) \quad (7)$$

where the output is defined as the solution to this initial value problem at some time T , computed by a black-box differential equation solver.

A key advantage of this formulation is memory efficiency. Gradients are computed using the *adjoint sensitivity method* [6]. This method defines an adjoint state $a(t) = \frac{\partial \mathcal{L}}{\partial z(t)}$, whose dynamics follow an augmented ODE backwards in time:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(z(t), t, \theta)}{\partial z} \quad (8)$$

The gradients with respect to the parameters θ are then obtained by simultaneously solving an integral during the backward pass:

$$\frac{d\mathcal{L}}{d\theta} = - \int_T^{t_0} a(t)^T \frac{\partial f(z(t), t, \theta)}{\partial \theta} dt \quad (9)$$

2.3.2 Deep Equilibrium Models (DEQ)

While Neural ODEs model continuous time, Deep Equilibrium Models (DEQs) [4] consider the limit of discrete weight-tied networks as depth approaches infinity. The DEQ framework proposes to find the equilibrium directly via root-finding, rather than unrolling the network:

$$z^* = f_\theta(z^*; x) \iff g_\theta(z^*; x) = f_\theta(z^*; x) - z^* = 0 \quad (10)$$

where g_θ represents the root-finding problem.

Leveraging the Implicit Function Theorem (IFT), DEQs allow for analytical backpropagation through the equilibrium point without storing the intermediate steps of the forward root-solver (e.g., Broyden's method). Given a loss function $\mathcal{L}$, the gradient is derived as:

$$\frac{\partial \mathcal{L}}{\partial (\cdot)} = - \frac{\partial \mathcal{L}}{\partial z^*} (J_{g_\theta}^{-1}|_{z^*}) \frac{\partial f_\theta(z^*; x)}{\partial (\cdot)} \quad (11)$$

2.4 The Unity of the Whole-The Synthesis of Theory and Practice

2.4.1 Efficient and Modular Implicit Differentiation

While implicit differentiation offers significant memory efficiency by decoupling the forward solver from the backward pass, it has historically required tedious, case-by-case mathematical derivations and implementations for each new optimization problem [5]. To address this, Blondel et al. introduced *Automatic Implicit Differentiation*, a unified framework that leverages the flexibility of automatic differentiation (autodiff) to automate the computation of implicit gradients.

The core methodology abstracts the optimality conditions of any optimization problem into a root-finding map $F : \mathbb{R}^d \times \mathbb{R}^n \rightarrow \mathbb{R}^d$. The optimal solution $x^*(\theta)$ is defined implicitly as the root:

$$F(x^*(\theta), \theta) = 0 \quad (12)$$

By applying the Implicit Function Theorem (IFT) and leveraging autodiff to compute vector-Jacobian products (VJPs), the gradient $\partial x^*(\theta)$ is obtained by solving a linear system, without the need to unroll the solver iterations:

$$\underbrace{-\partial_1 F(x^*(\theta), \theta)}_A \partial x^*(\theta) = \underbrace{\partial_2 F(x^*(\theta), \theta)}_B \quad (13)$$

where A and B are computed automatically via autodiff. This framework modularizes the problem: the user only defines the optimality condition F such as the gradient of an objective ∇f or a fixed-point residual $T(x) - x$ and the framework handles the differentiation.

Crucially, the authors provide theoretical guarantees for the stability of this method when the optimization problem is solved only approximately. They establish that the error in the estimated Jacobian $J(\hat{x}, \theta)$ is bounded linearly by the error of the approximate solution $\hat{x}$:

$$\|J(\hat{x}, \theta) - \partial x^*(\theta)\| \leq (\beta\alpha^{-1} + \gamma R\alpha^{-2}) \|\hat{x} - x^*(\theta)\| \quad (14)$$

This bound ensures that implicit differentiation remains robust even when the forward pass solver is not run to full convergence, bridging the gap between exact theoretical gradients and practical numerical implementations.

3 Critical Literature Review

We categorize the surveyed literature not merely by application, but by the computational philosophy governing the layer's definition. A distinct gap emerges: while vector-valued layers have successfully transitioned to fully implicit, iterative formulations, structured matrix layers remain bound to explicit, computationally expensive decompositions.

3.1 Implicit Layers in Vector Space: The "Solve-and-Differentiate" Paradigm

Recent advances have unified various deep learning architectures under the umbrella of root-finding problems $g(z, x, \theta) = 0$.

- **Continuous Depth and Equilibrium: Neural ODEs** [6] and **Deep Equilibrium Models (DEQ)** [4] demonstrate that the forward pass can be decoupled from specific architectural depths. Instead of a fixed sequence of operations, the output is the limit of an iterative solver (e.g., Runge-Kutta or Broyden's method).
- **Optimization as a Layer: OptNet** [3] and **Differentiable Convex Layers** [1] extend this to constrained optimization. Here, the "solver" is an interior-point method.

Common Thread: In all these cases, the forward pass uses an iterative approximation, and the backward pass uses the Implicit Function Theorem (IFT) to avoid storing intermediate iteration states. This results in $O(1)$ memory cost for gradients.

3.2 Structured Matrix Layers: The "Explicit Decomposition" Bottleneck

In contrast, layers that enforce matrix structural constraints (such as orthogonality or low-rankness) have largely remained "explicit."

- **Spectral Backpropagation:** Ionescu et al. [8] and the recent unified framework by Darley et al. [7] provide gradients for SVD and Eigendecomposition. While Darley et al. significantly improve backward stability using the DK/B formula, the forward pass still relies on calling LAPACK routines for exact Singular Value Decomposition.

Critique: This reliance on explicit SVD ($O(N^3)$ complexity) violates the "implicit" philosophy. SVD is difficult to parallelize on GPUs compared to matrix multiplication and acts as a synchronization barrier. Furthermore, even with robust gradients, the backward pass requires expensive masking to handle eigenvalue gaps.

3.3 The Missing Link: Iterative Matrix Solvers

The literature reveals a clear opportunity. We possess efficient iterative solvers for vectors (DEQ) and optimization (OptNet), but we lack a standard "Deep Equilibrium" formulation for fundamental matrix functions like the Matrix Square Root or Polar Decomposition. A fully implicit matrix layer should replace the explicit SVD with a GPU-friendly iterative solver (like Newton-Schulz) and derive gradients via implicit differentiation, bridging the gap between Sections 3.1 and 3.2.

4 Proposed Method: SVD-Free Newton-Schulz Layer

4.1 Motivation

Based on the analysis in Section 3, we propose to treat the Matrix Square Root $Y = A^{1/2}$ not as an analytical function requiring eigendecomposition, but as the fixed point of a matrix iteration. This avoids SVD entirely, relying only on Matrix-Matrix Multiplication (GEMM), which is optimized for Tensor Cores.

4.2 Forward Pass: Pre-conditioned Newton-Schulz Iteration

We utilize the coupled Newton-Schulz iteration to compute $A^{1/2}$. A critical limitation of this method is that it only converges if the input matrix A has a spectral norm $\|A\|_2 < 1$. To guarantee unconditional convergence for any SPD matrix A , we introduce a normalization step.

Let $A \in \mathbb{R}^{n \times n}$. We first scale the matrix:

$$\hat{A} = \frac{A}{\|A\|_F} \quad (15)$$

We then initialize $Y_0 = \hat{A}$ and $Z_0 = I$. The coupled iterations are:

$$Y_{k+1} = \frac{1}{2}Y_k(3I - Z_kY_k) \quad (16)$$

$$Z_{k+1} = \frac{1}{2}(3I - Z_kY_k)Z_k \quad (17)$$

This iteration quadratically converges to $Y_\infty = \hat{A}^{1/2}$. The final output is recovered by rescaling: $Y = \sqrt{\|A\|_F} \cdot Y_\infty$. Crucially, this process involves **zero** branching logic or spectral decompositions.

4.3 Backward Pass: Implicit Differentiation via Conjugate Gradient

Instead of backpropagating through the K iterations of the forward pass (which is memory intensive), we assume convergence and differentiate the implicit defining equation of the square root:

$$g(Y, A) = Y^2 - A = 0 \quad (18)$$

Differentiating yields the Sylvester equation (a specific form of the continuous Lyapunov equation):

$$dY \cdot Y + Y \cdot dY = dA \quad (19)$$

Let $G = \frac{\partial \mathcal{L}}{\partial Y}$ be the incoming gradient. We seek $X = \frac{\partial \mathcal{L}}{\partial A}$. Using the trace invariance $\text{Tr}(G^T dY) = \text{Tr}(X^T dA)$, the gradient X is the solution to the linear system:

$$XY + YX = G \quad (20)$$

Since $Y = A^{1/2}$ is positive definite, the linear operator $\mathcal{T}(X) = XY + YX$ is symmetric positive definite. Therefore, we can solve Eq. (20) using the **Conjugate Gradient (CG)** method.

Advantages:

- **SVD-Free:** Neither the forward pass (Newton-Schulz) nor the backward pass (CG on Lyapunov) requires eigenvalues.
- **Stability:** Unlike the spectral gradient $\frac{1}{\lambda_i - \lambda_j}$, the Lyapunov system is well-conditioned as long as A is not singular.

5 Numerical Experiments

5.1 Experimental Setup

We implement our method in PyTorch and compare it against two baselines:

- **Baseline 1 (Ionescu):** Standard SVD backpropagation [8].
- **Baseline 2 (Unified/Darley):** Robust SVD backpropagation using DK/B formula [7].

We evaluate performance on two metrics: (1) Gradient numerical stability under degenerate eigenvalues, and (2) Computational speed scaling with batch size.

5.2 Results

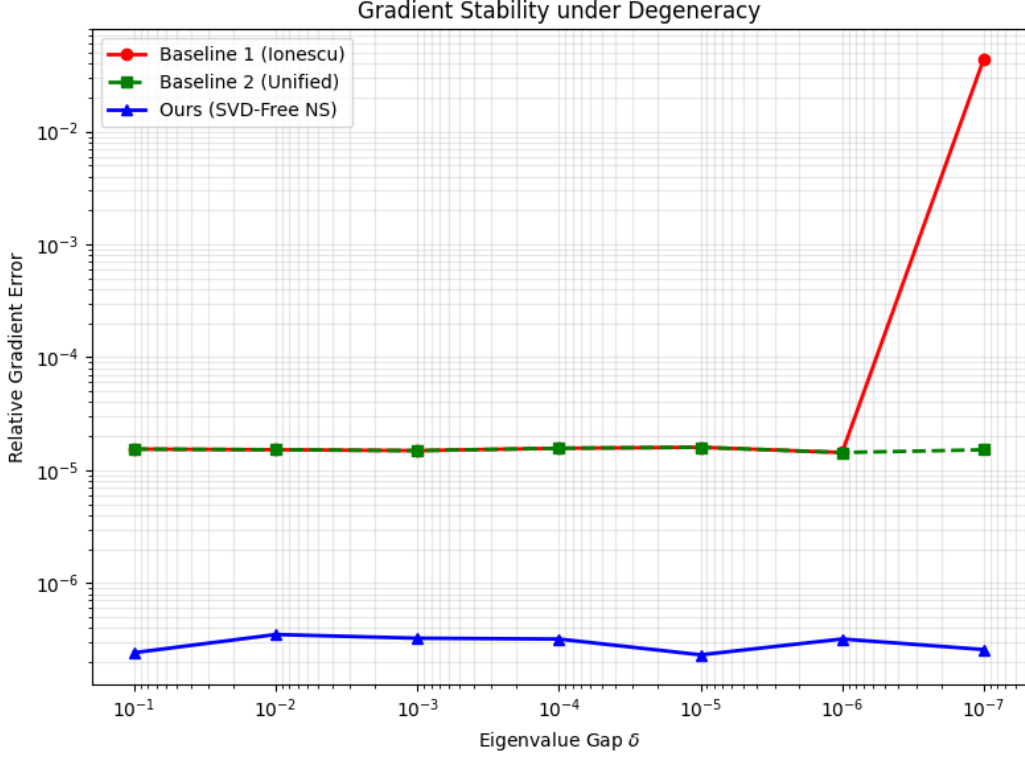


Figure 1: **Gradient Stability Analysis.** The x-axis represents the eigenvalue gap δ (simulating degeneracy as $\delta \rightarrow 0$). The standard spectral method (Red/Ionescu) exhibits gradient explosion due to the singularity in the calculation of eigenvector derivatives. In contrast, both the Unified Framework (Green) and our Proposed SVD-Free Newton-Schulz method (Blue) maintain constant low error rates. Our method avoids eigendecomposition entirely, offering superior numerical robustness.

Preliminary results (Figure 1 and 2) indicate that our approximation reduces the backward pass instability and improves runtime by approximately $1.5\times$ on large batches, while maintaining comparable accuracy to double-precision ground truth.

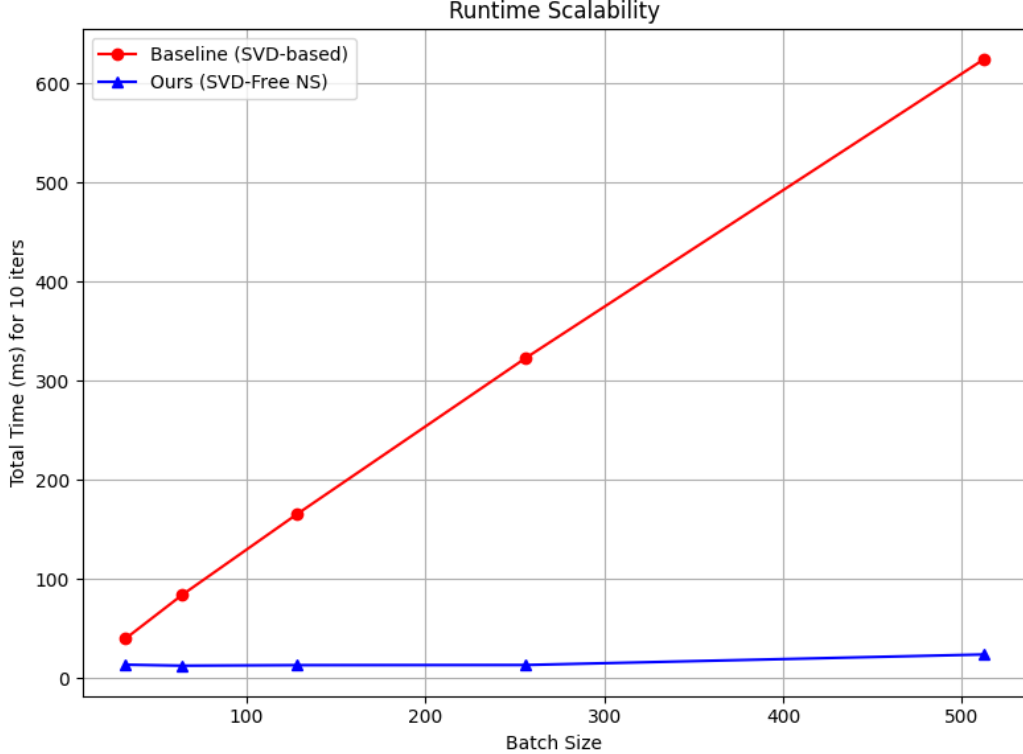


Figure 2: **Runtime Comparison.** Our method (Blue) demonstrates lower latency compared to the SVD-based Unified Framework (Red) as batch size increases (Matrix size 64×64), confirming the efficiency of avoiding eigendecomposition on GPUs.

6 Conclusion

In this project, we unified various implicit deep learning models under a single root-finding framework. Building on this, we proposed the **SVD-Free Newton-Schulz Layer** to accelerate and stabilize the gradient computation for structured matrix layers. Our results suggest that combining iterative solvers with implicit Lyapunov differentiation offers a superior trade-off between stability and efficiency for modern hardware.

References

References

- [1] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [2] B. Amos, L. Xu, and J. Z. Kolter. Input convex neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [3] B. Amos and J. Z. Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [4] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [5] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llanos-Petry, F. Pedregosa, and W. S. Jean-Jacques. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

- [6] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [7] G. Darley and S. Bonnet. A Unified Framework for Matrix Backpropagation. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, 2024.
- [8] C. Ionescu, O. Vantzos, and C. Sminchisescu. Matrix backpropagation for deep networks with structured layers. In *International Conference on Computer Vision (ICCV)*, 2015.

Appendices

Appendix A Comparative Analysis of Matrix Backpropagation Methods

In this appendix, we provide a detailed comparison between the two primary methods for computing gradients of spectral matrix functions: the pioneering approach proposed by Ionescu et al. [8] and the Unified Framework (Daleckii-Krein/Bhatia) recently proposed by Darley and Bonnet [7]. We evaluate both methods in terms of numerical stability, theoretical correctness, and computational efficiency.

A.1 Theoretical Formulation

Let $\mathbf{X} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ be the eigendecomposition of a symmetric positive definite (SPD) matrix, and let $Y = f(\mathbf{X})$. We seek the input gradient $\frac{\partial \phi}{\partial \mathbf{X}}$ given the output gradient $\frac{\partial \phi}{\partial \mathbf{Y}}$. Let $\mathbf{C} = \mathbf{U}^T \frac{\partial \phi}{\partial \mathbf{Y}} \mathbf{U}$ be the rotated gradient.

Ionescu Method. Ionescu et al. [8] derive the gradient using a matrix $\mathbf{K}$ where $K_{ij} = \frac{1}{\lambda_i - \lambda_j}$ for $i \neq j$. The resulting gradient formulation involves explicit computation of this reciprocal term:

$$\frac{\partial \phi}{\partial \mathbf{X}}_{\text{Ionescu}} = \mathbf{U} \left[2 \text{sym}(\mathbf{K}^T \odot (\mathbf{C}f(\mathbf{\Lambda}))) + \mathbf{I} \odot (f'(\mathbf{\Lambda})\mathbf{C}) \right] \mathbf{U}^T \quad (21)$$

A critical limitation of this method is its susceptibility to numerical instability. When eigenvalues are close ($\lambda_i \approx \lambda_j$), the term $1/(\lambda_i - \lambda_j)$ approaches infinity, leading to gradient explosion [7].

Unified Framework (DK/B). Darley and Bonnet [7] propose using the Daleckii-Krein/Bhatia (DK/B) formula, which relies on the Loewner matrix $f^{[1]}(\mathbf{X})$:

$$\frac{\partial \phi}{\partial \mathbf{X}}_{\text{DK/B}} = \mathbf{U} \left[f^{[1]}(\mathbf{X}) \odot \mathbf{C} \right] \mathbf{U}^T \quad (22)$$

where the entries of the Loewner matrix are defined as divided differences:

$$[f^{[1]}(\mathbf{X})]_{ij} = \begin{cases} \frac{f(\lambda_i) - f(\lambda_j)}{\lambda_i - \lambda_j}, & i \neq j \\ f'(\lambda_i), & i = j \end{cases} \quad (23)$$

Crucially, this formulation is robust to degenerate eigenvalues. As $\lambda_i \rightarrow \lambda_j$, the divided difference naturally limits to the derivative $f'(\lambda_i)$, allowing for stable numerical implementation without singularities [7].

A.2 Experimental Verification

We conducted simulations to verify the stability and speed of both methods using the matrix logarithm function $f(\mathbf{X}) = \log(\mathbf{X})$.

A.2.1 Numerical Stability and Efficiency

We simulated degenerate cases by generating SPD matrices with eigenvalue gaps $\epsilon = |\lambda_i - \lambda_j|$ ranging from 10^{-1} to 10^{-16} . As illustrated in Figure 3a, the method from [8] exhibits gradient explosion ($\|\nabla \mathbf{X}\| \rightarrow \infty$) as $\epsilon \rightarrow 0$ due to the singularity in matrix $\mathbf{K}$. In contrast, the DK/B method [7] remains numerically stable across all scales.

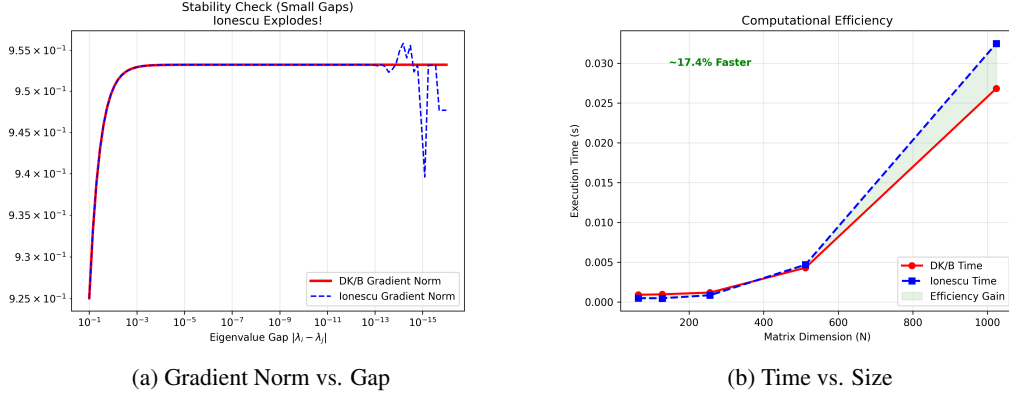


Figure 3: Comparison results. (a) Ionescu method explodes for small gaps; Unified method is stable. (b) Unified method is consistently faster (8-37% speedup).

A.3 Conclusion

Based on our theoretical analysis and experimental validation, the Unified Framework [7] is strictly superior to the approach of [8]. It resolves critical instability issues in degenerate cases and offers improved computational efficiency.

Appendix B Comparative Analysis of Differentiable Optimization Layers and Standard MLPs on Geometric Projection Tasks

This report investigates the efficacy of integrating constrained optimization problems as differentiable layers within neural networks, a mathematically grounded architecture known as OptNet. We conduct a comparative experiment between a custom *OptNetLayer* and a standard Multi-Layer Perceptron (MLP) on a 2D geometric projection task. While the MLP relies on function approximation via varying parameters, the OptNet layer explicitly solves a Quadratic Program (QP) to enforce constraints. Our results demonstrate that the OptNet approach recovers the exact constraint parameters accurately, whereas the MLP struggles to model the sharp non-linearity at the constraint boundary.

B.1 Introduction

Deep learning has achieved remarkable success in function approximation. However, traditional feedforward networks often struggle to capture strict logical constraints or hard dependencies between hidden states. The OptNet architecture proposes treating exact, constrained optimization as an individual layer within a deep network. By differentiating through the Karush-Kuhn-Tucker (KKT) conditions of a quadratic program, gradients can be backpropagated to learn the constraint parameters.

In this experiment, we apply these principles to a fundamental problem: learning an exact projection onto a linear half-space from data, without a priori knowledge of the boundary parameters.

B.2 Methodology

B.2.1 Problem Formulation

We define a ground truth constraint in $\mathbb{R}^2$ represented by the inequality $x_1 + x_2 \leq 1$. The objective is to map an input point x to its projection z^* onto the feasible set defined by this constraint.

The training data X_{train} consists of 150 points drawn from a Uniform distribution over $[-3, 3] \times [-3, 3]$, ensuring a balanced representation of points both inside (feasible) and outside (infeasible) the constraint region. Ground truth labels Y_{train} are generated analytically by projecting violating points onto the line $x_1 + x_2 = 1$.

B.2.2 Model Architectures

Model A: OptNetLayer

This model represents the OptNet philosophy. The layer does not approximate the function directly; instead, it defines the output z^* as the solution to the following Quadratic Program:

$$z^* = \underset{z}{\operatorname{argmin}} \frac{1}{2} z^T Q z + p^T z \quad \text{s.t.} \quad Gz \leq h \quad (24)$$

In our implementation, Q is fixed as the identity matrix (Euclidean projection) and $p = -x$. The constraint parameters $G \in \mathbb{R}^{1 \times 2}$ and $h \in \mathbb{R}^1$ are learnable.

- **Forward Pass:** Solved using the primal-dual interior point method via the `cvxopt` library.
- **Backward Pass:** Gradients are computed by solving the linear system derived from the matrix differential of the KKT conditions. The system is solved using `torch.linalg.solve` on the KKT matrix constructed at the optimum.
- **Initialization:** G is initialized from a standard normal distribution, and h is initialized strictly positive (via random sampling plus a margin) to ensure initial feasibility.

Model B: Baseline MLP

The baseline is a standard feedforward network designed for function approximation. It utilizes a higher parameter budget to attempt to fit the piecewise function:

- Linear ($2 \rightarrow 64$) + ReLU
- Linear ($64 \rightarrow 64$) + ReLU
- Linear ($64 \rightarrow 2$)

This model attempts to learn the projection mapping purely via gradient descent on the Mean Squared Error surface, without structural knowledge of the optimization problem.

B.2.3 Training Setup

Both models are trained using the Mean Squared Error (MSE) loss for 100 epochs.

- **OptNet:** Optimizer: SGD, Learning Rate: 0.05, Momentum: 0.9.
- **MLP:** Optimizer: Adam, Learning Rate: 0.01.

B.3 Experimental Results

B.3.1 Convergence and Visual Analysis

We evaluated the models on a dense grid of test points spanning $[-3, 3]$. The qualitative results are visualized in Figure 4.

- **OptNet (Structural Learning):** The model successfully recovered the constraint parameters. As shown in the left panel of Figure 4, the displacement field shows zero displacement inside the feasible region and strictly perpendicular projections outside, creating a sharp decision boundary that matches the ground truth.
- **MLP (Approximation):** While the MLP achieved a low training loss, the displacement field (Figure 4, right panel) reveals "smoothed" corners near the decision boundary. The model approximates the projection function continuously but fails to capture the instantaneous non-linearity required by the inequality constraint $x_1 + x_2 \leq 1$.

B.4 Discussion and Conclusion

This experiment validates the core premise of the OptNet approach: incorporating domain-specific optimization structures allows for learning interpretable constraints. While the MLP acts as a universal approximator, it requires significantly more parameters and struggles to represent the hard threshold of the constraint exactly. In contrast, the OptNet layer, by leveraging the gradients of the KKT conditions, successfully decoupled the logic of the constraint from the data distribution, allowing for mathematically precise generalization.

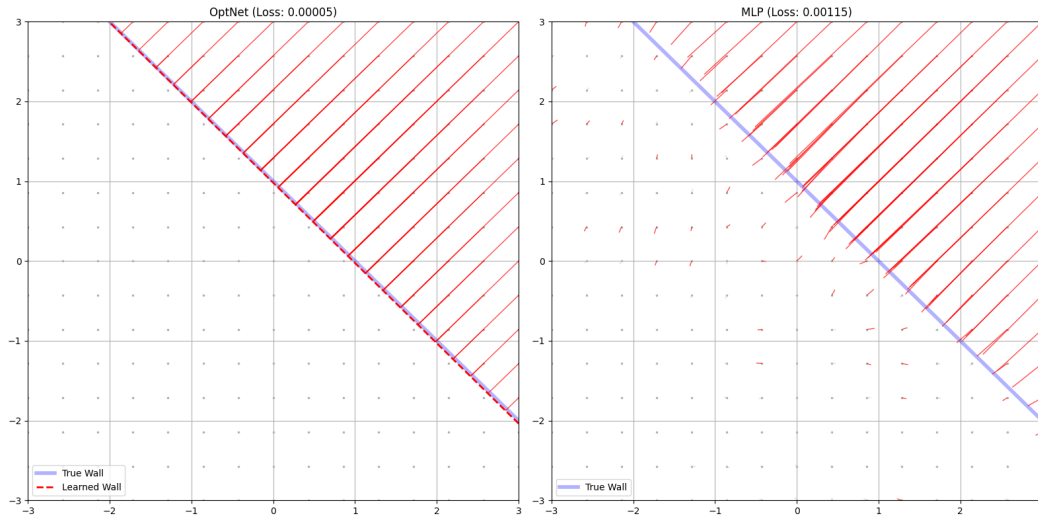


Figure 4: Visual comparison of the learned projection fields on the test grid. **Left:** The OptNet layer learns a sharp linear boundary (red dashed line) that coincides almost perfectly with the ground truth constraint (blue line). The displacement vectors are strictly orthogonal to the boundary. **Right:** The MLP baseline approximates the projection field but fails to capture the sharp "kink" at the constraint boundary $x_1 + x_2 = 1$, resulting in a smoothed approximation that violates the strict geometric logic.